

Readora

Bookstore Platform — Technical Design & Architecture

Version 4.0 · 20 August 2026

9 services · PostgreSQL + pgvector · Redis · Kafka (KRaft) · WebSocket · React

AI: Spring AI agent · RAG · MCP · gpt-oss-120b on AWS Bedrock

Observability: OpenTelemetry → OTel Collector → New Relic

BACKEND

Spring Boot

Spring Cloud Gateway

Spring AI

Java 21

FRONTEND

React

TypeScript

DATA

PostgreSQL

pgvector

Redis

MESSAGING

Apache Kafka

AI

AWS Bedrock

gpt-oss-120b

MCP

OBSERVABILITY

OpenTelemetry

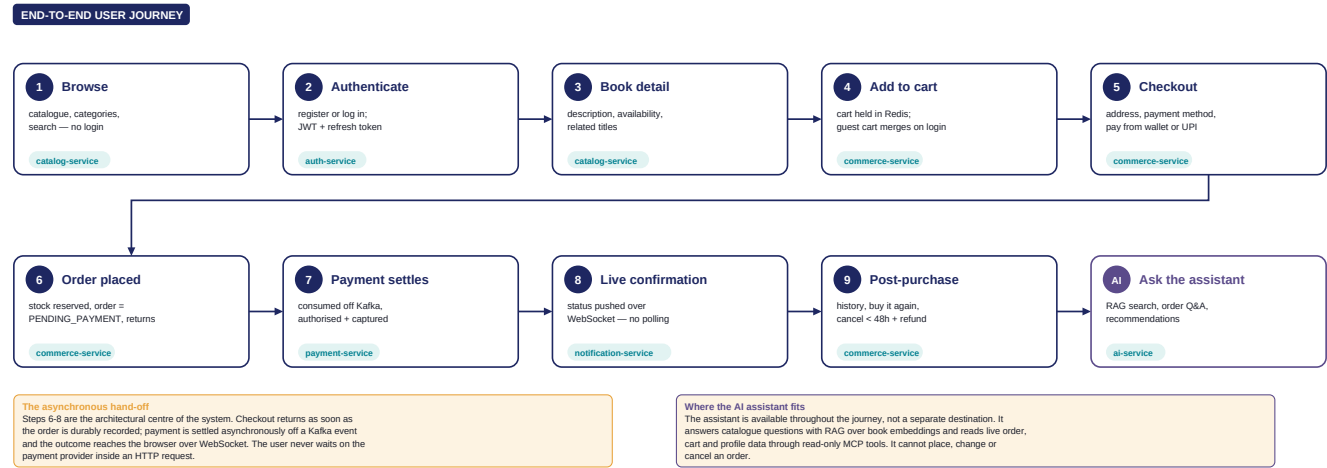
New Relic

CI / CONTAINERS

Docker

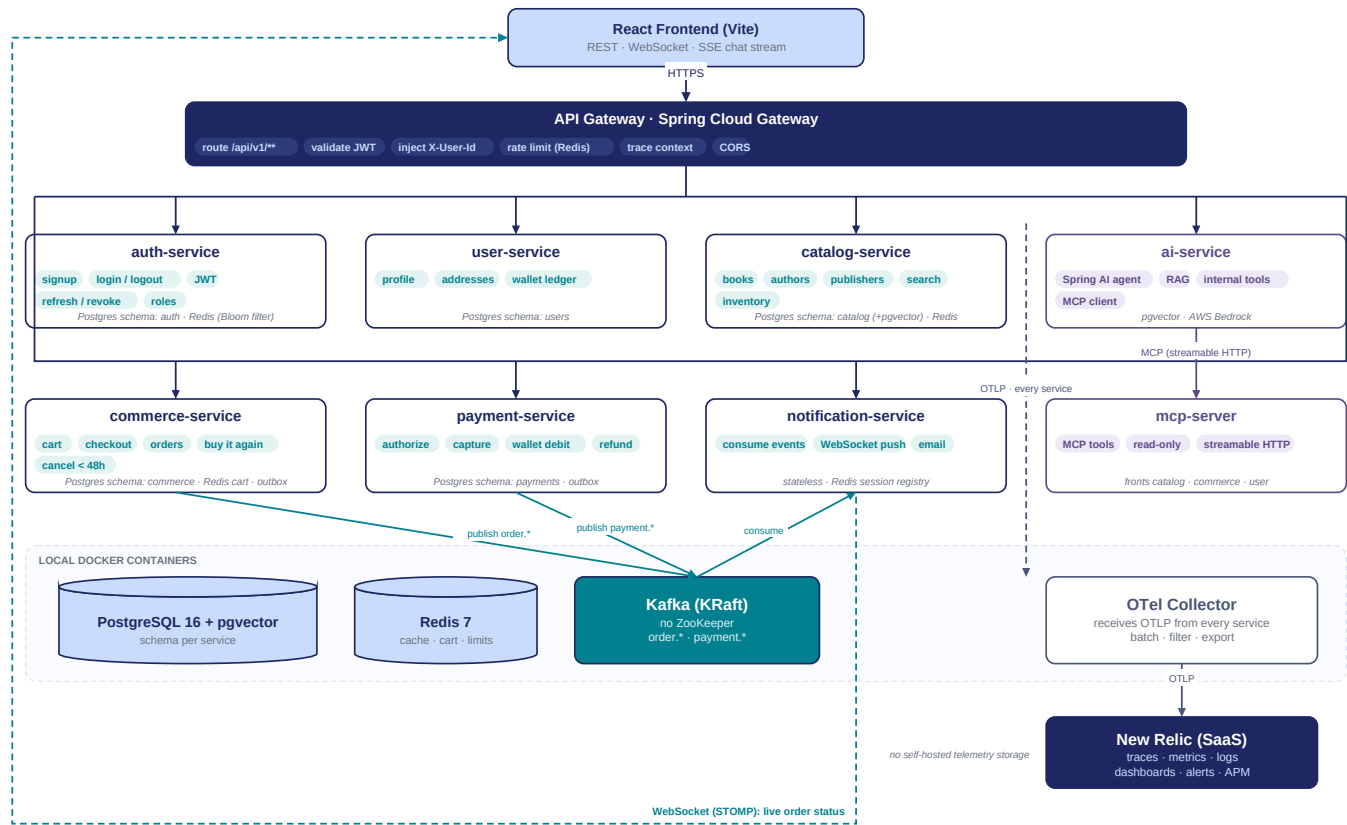
GitHub Actions

Application Flow



The end-to-end user journey and the service that owns each step.

System Architecture



Services, responsibilities, local container environment and telemetry export.

Local Environment

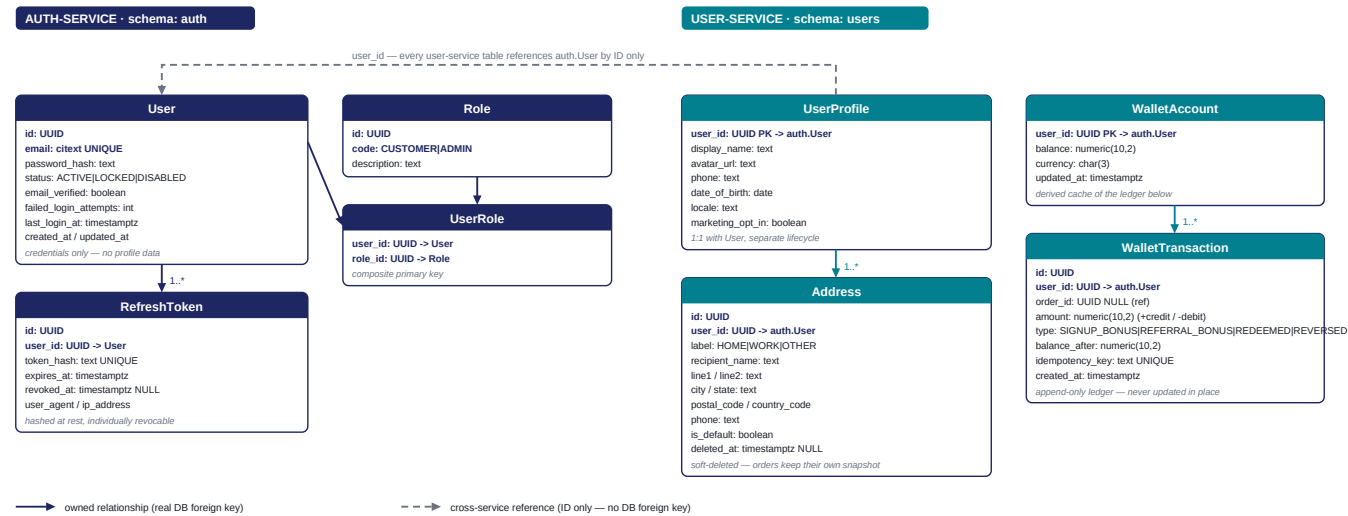
Every dependency runs as a Docker container defined in `infra/docker-compose.yml`. A clean clone plus docker compose up produces the whole system.



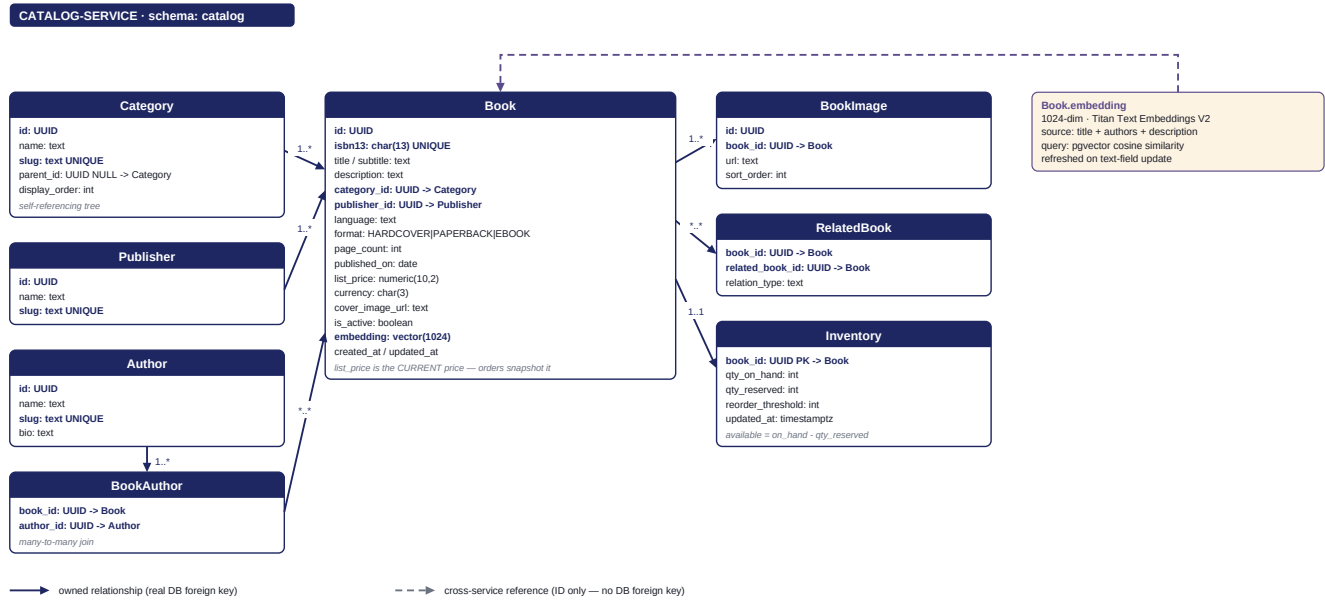
Two things that are deliberately not containers
New Relic is a SaaS platform — there is no self-hosted telemetry storage to run or operate (see Observability).
AWS Bedrock hosts the model and the embedding endpoint — `ai-service` calls it with AWS credentials; no model runs locally (see AI Service Architecture).

Domain Model

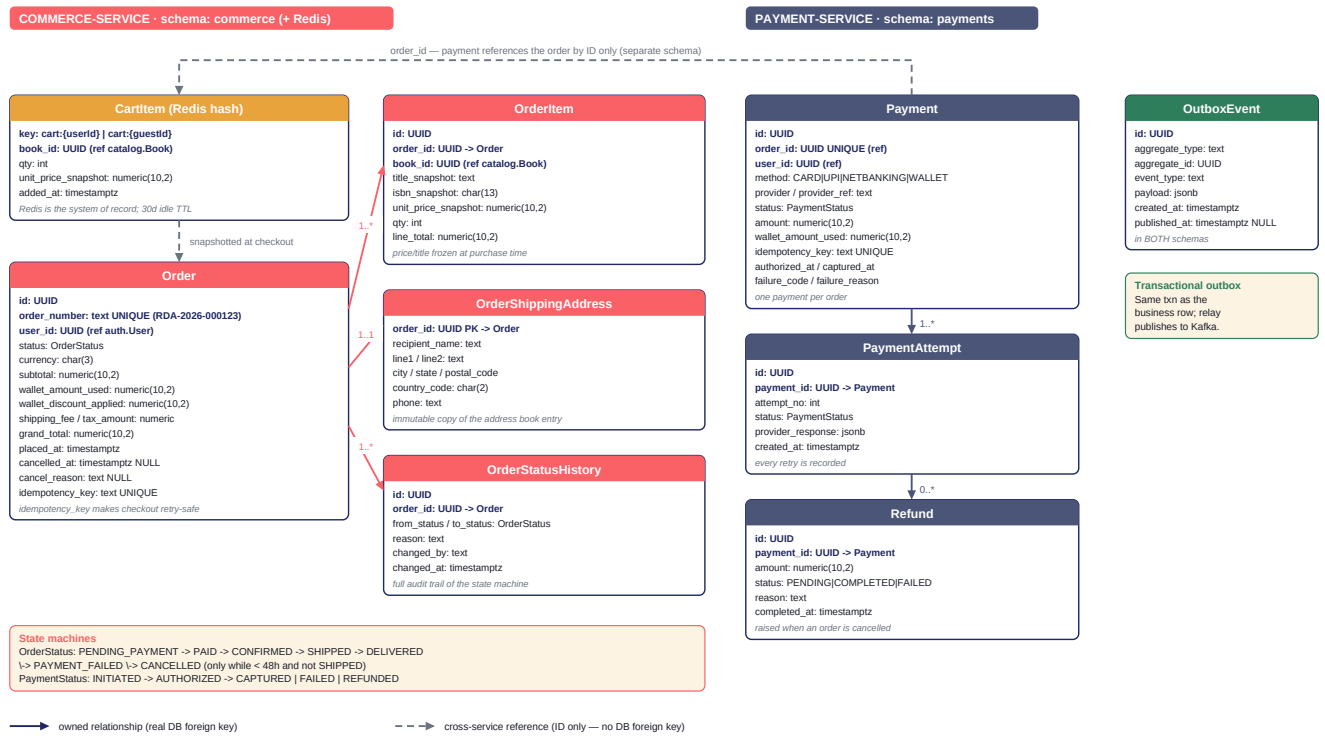
Each service owns its tables outright. There are **no cross-service foreign keys** — a service referencing another service's entity stores only the bare id, treated as opaque.



Identity: auth-service (credentials) and user-service (profile, addresses, wallet).



Catalogue, inventory and the pgvector embedding column used for RAG.



Commerce (cart + orders) and payments, including the transactional outbox.

API Reference

All endpoints are served through **api-gateway** at `http://localhost:8080` under `/api/v1`. **Auth** column: **Public** = no token; **Bearer** = valid access token required; **Refresh** = refresh token required.

auth-service

Login, signup, JWT issue, roles, refresh and revoke.

POST /api/v1/auth/register

Public

Creates an account. The email is checked against a Redis Bloom filter before Postgres is touched.

Request body

```
{
  "email": "pavan@example.com",
  "password": "S3cure!passphrase",
  "fullName": "Pavan B N"
}
```

Success — 201 Created

```
{
  "userId": "0190c3f2-7b41-7c2a-9f10-6d2a1c8e4b55",
  "email": "pavan@example.com",
  "createdAt": "2026-08-19T10:15:00Z"
}
```

Error responses

Status	Error code	When it happens
400	VALIDATION_FAILED	Malformed email, or password below policy (min 10 chars). Includes <code>fieldErrors</code> .
409	EMAIL_ALREADY_REGISTERED	An account already exists for this email.

POST /api/v1/auth/login

Public

Exchanges credentials for an access token and a refresh token.

Request body

```
{
  "email": "pavan@example.com",
  "password": "S3cure!passphrase"
}
```

Success — 200 OK

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
  "refreshToken": "9f8e7d6c-5b4a-...",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

Error responses

Status	Error code	When it happens
401	INVALID_CREDENTIALS	Email unknown or password incorrect — deliberately the same code for both.
423	ACCOUNT_LOCKED	Too many consecutive failed attempts.

Unknown-email and wrong-password return an identical response so the endpoint cannot be used to enumerate registered accounts.

POST **/api/v1/auth/refresh**

Refresh

Rotates the refresh token and issues a new access token. The presented token is revoked as the new pair is issued.

Request body

```
{ "refreshToken": "9f8e7d6c-5b4a-..." }
```

Success — 200 OK

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
  "refreshToken": "1a2b3c4d-...",
  "expiresIn": 900
}
```

Error responses

Status	Error code	When it happens
401	REFRESH_TOKEN_INVALID	Unknown, expired, or already-revoked token.
401	REFRESH_TOKEN_REUSED	A revoked token was replayed — all sessions for the user are revoked.

POST **/api/v1/auth/logout**

Bearer

Revokes the supplied refresh token. The access token remains valid until it expires (max 15 minutes).

Success — 204 No Content

(empty body)

user-service

Profile, address book, wallet ledger.

GET **/api/v1/users/me**

Bearer

Returns the caller's profile together with their wallet balance.

Success — 200 OK

```
{
  "userId": "0190c3f2-...",
  "email": "pavan@example.com",
  "displayName": "Pavan",
  "avatarUrl": "https://.../avatars/0190c3f2.png",
  "locale": "en-IN",
  "wallet": { "balance": "340.00", "currency": "INR" }
}
```

GET `/api/v1/users/me/addresses`

Bearer

Lists the caller's non-deleted addresses, default first.

Success — 200 OK

```
{
  "items": [
    {
      "id": "b21f...", "label": "HOME", "recipientName": "Pavan B N",
      "line1": "12 MG Road", "city": "Bengaluru", "state": "KA",
      "postalCode": "560001", "countryCode": "IN", "isDefault": true
    }
  ]
}
```

POST `/api/v1/users/me/addresses`

Bearer

Adds an address to the caller's address book.

Request body

```
{
  "label": "WORK",
  "recipientName": "Pavan B N",
  "line1": "44 Residency Road", "line2": "Level 3",
  "city": "Bengaluru", "state": "KA",
  "postalCode": "560025", "countryCode": "IN",
  "phone": "+91...", "isDefault": false
}
```

Success — 201 Created

```
{ "id": "c93a...", "isDefault": false }
```

Error responses

Status	Error code	When it happens
400	VALIDATION_FAILED	Missing required address fields, or unknown country code.
409	ADDRESS_LIMIT_REACHED	The account already holds the maximum of 20 addresses.

DELETE `/api/v1/users/me/addresses/{id}`

Bearer

Soft-deletes an address. Orders that shipped to it keep their own immutable copy.

Success — 204 No Content

(empty body)

Error responses

Status	Error code	When it happens
404	ADDRESS_NOT_FOUND	No such address belonging to the caller.

GET `/api/v1/users/me/wallet`

Bearer

Returns the wallet balance and a paginated ledger of wallet transactions.

Success — 200 OK

```
{
  "balance": "340.00", "currency": "INR",
  "items": [
    { "id": "e11c...", "amount": "-200.00", "type": "REDEEMED",
      "balanceAfter": "340.00", "orderId": "8f2a...",
      "createdAt": "2026-08-19T10:15:04Z" },
    { "id": "d02b...", "amount": "+200.00", "type": "REFERRAL_BONUS",
      "balanceAfter": "540.00", "orderId": null,
      "createdAt": "2026-08-10T09:00:00Z" },
    { "id": "a90f...", "amount": "+100.00", "type": "SIGNUP_BONUS",
      "balanceAfter": "100.00", "orderId": null,
      "createdAt": "2026-08-02T09:00:00Z" }
  ]
}
```

The ledger is append-only. A reversal appears as an additional REVERSED row, never as an edit to an existing one. New accounts start with a 100.00 SIGNUP_BONUS; a successful referral credits 200.00 to the referrer.

catalog-service

Books, authors, publishers, categories, search, inventory, embeddings.

GET	/api/v1/books	Public
-----	---------------	--------

Searches and filters the catalogue. Query params: q (free text over title/author), categoryId, publisherId, format, minPrice, maxPrice, sort, page, size.

Success — 200 OK

```
{
  "items": [
    {
      "id": "4c7e...", "isbn13": "9780132350884",
      "title": "Clean Code", "authors": ["Robert C. Martin"],
      "publisher": "Prentice Hall", "format": "PAPERBACK",
      "listPrice": "38.50", "currency": "USD",
      "coverImageUrl": "https://.../4c7e.jpg",
      "availability": "IN_STOCK"
    }
  ],
  "page": 0, "size": 20, "totalElements": 137, "totalPages": 7
}
```

Error responses

Status	Error code	When it happens
400	INVALID_QUERY_PARAM	Unparseable price range, unknown sort field, or size above 100.

Responses are cached in Redis (cache-aside, 10-minute TTL) and invalidated on any write to the book.

GET	/api/v1/books/{id}	Public
-----	--------------------	--------

Full detail for one book, including description, live availability and the estimated delivery window.

Success — 200 OK

```
{
  "id": "4c7e...", "isbn13": "9780132350884",
  "title": "Clean Code",
  "subtitle": "A Handbook of Agile Software Craftsmanship",
  "description": "...",
  "authors": [{ "id": "a1...", "name": "Robert C. Martin" }],
  "category": { "id": "c9...", "name": "Software Engineering" },
  "publisher": { "id": "p3...", "name": "Prentice Hall" },
  "format": "PAPERBACK", "pageCount": 464, "language": "en",
  "publishedOn": "2008-08-01",
  "listPrice": "38.50", "currency": "USD",
  "images": ["https://.../4c7e-1.jpg", "https://.../4c7e-2.jpg"],
  "availability": { "status": "IN_STOCK", "quantityAvailable": 23 },
  "estimatedDeliveryDays": 3
}
```

Error responses

Status	Error code	When it happens
404	BOOK_NOT_FOUND	No active book with that id. A Bloom filter absorbs repeated lookups for ids that have never existed.

GET **/api/v1/books/{id}/related**

Public

Related titles for cross-sell on the detail page.

Success — 200 OK

```
{ "items": [ { "id": "8b2f...", "title": "The Pragmatic Programmer",
  "listPrice": "42.00", "coverImageUrl": "..." } ] }
```

Error responses

Status	Error code	When it happens
404	BOOK_NOT_FOUND	The parent book does not exist.

GET **/api/v1/categories**

Public

Returns the category tree (parents with nested children).

Success — 200 OK

```
{
  "items": [
    { "id": "c1...", "name": "Technology", "slug": "technology",
      "children": [ { "id": "c9...", "name": "Software Engineering",
        "slug": "software-engineering" } ] }
  ]
}
```

commerce-service — cart

Cart, checkout and orders.

GET **/api/v1/cart**

Bearer

Returns the caller's cart with line totals recomputed against current catalogue prices.

Success — 200 OK

```
{
  "items": [
    { "bookId": "4c7e...", "title": "Clean Code", "qty": 2,
      "unitPrice": "38.50", "lineTotal": "77.00",
      "priceChanged": false, "availability": "IN_STOCK" }
  ],
  "subtotal": "77.00", "currency": "USD", "itemCount": 2
}
```

priceChanged is true when the catalogue price has moved since the item was added, so the UI can warn before checkout.

POST **/api/v1/cart/items**

Bearer

Adds a book to the cart, or increments its quantity if already present.

Request body

```
{ "bookId": "4c7e...", "qty": 1 }
```

Success — 200 OK

```
{ "itemCount": 3, "subtotal": "115.50", "currency": "USD" }
```

Error responses

Status	Error code	When it happens
404	BOOK_NOT_FOUND	The book does not exist or is no longer active.
409	INSUFFICIENT_STOCK	Requested quantity exceeds available inventory.
422	QTY_LIMIT_EXCEEDED	More than 10 of a single title.

PUT **/api/v1/cart/items/{bookId}**

Bearer

Sets the absolute quantity of a line item. A quantity of 0 removes it.

Request body

```
{ "qty": 3 }
```

Success — 200 OK

```
{ "itemCount": 4, "subtotal": "154.00", "currency": "USD" }
```

Error responses

Status	Error code	When it happens
404	CART_ITEM_NOT_FOUND	That book is not in the cart.
409	INSUFFICIENT_STOCK	Requested quantity exceeds available inventory.

POST **/api/v1/cart/merge**

Bearer

Merges a guest cart into the authenticated user's cart after login. Quantities are summed and capped at the per-title limit.

Request body

```
{ "guestCartId": "g-7f21a9c4-..." }
```

Success — 200 OK

```
{ "itemCount": 5, "subtotal": "192.50", "merged": 2 }
```

Error responses

Status	Error code	When it happens
404	GUEST_CART_NOT_FOUND	The guest cart expired or never existed.

commerce-service — checkout & orders**POST** **/api/v1/orders/checkout**

Bearer

Reserves stock, prices the order, and creates it in PENDING_PAYMENT. Returns as soon as the order is durably recorded — payment happens asynchronously and the outcome arrives over WebSocket. Send an Idempotency-Key header; a replay returns the original order rather than creating a second one.

Request body

Idempotency-Key: 3f9a1c22-8d40-4e77-b0a1-9c2e5d7b1f03

```
{
  "addressId": "b21f...",
  "paymentMethod": "CARD",
  "walletAmountToUse": "50.00",
  "items": [ { "bookId": "4c7e...", "qty": 2 } ]
}
```

Success — 201 Created

```
{
  "orderId": "8f2a...",
  "orderNumber": "RDA-2026-000123",
  "status": "PENDING_PAYMENT",
  "subtotal": "77.00",
  "walletAmountUsed": "50.00",
  "walletDiscountApplied": "5.00",
  "shippingFee": "0.00",
  "taxAmount": "6.93",
  "grandTotal": "28.93",
  "currency": "USD",
  "placedAt": "2026-08-19T10:15:00Z"
}
```

Error responses

Status	Error code	When it happens
400	VALIDATION_FAILED	Empty items array or missing address.

Status	Error code	When it happens
404	ADDRESS_NOT_FOUND	The address id does not belong to the caller.
409	CART_EMPTY	Nothing to check out.
409	INSUFFICIENT_STOCK	A title went out of stock between cart and checkout. Body names the offending bookId.
409	PRICE_CHANGED	A price moved since the cart was priced; the client must re-confirm.
422	INSUFFICIENT_WALLET_BALANCE	Requested wallet amount exceeds the current balance.

A 201 means the order exists, not that payment succeeded. Terminal status arrives via WebSocket or by polling GET /orders/{id}. Paying (partly) from wallet applies a 10% discount on the wallet-funded portion.

GET /api/v1/orders

Bearer

Paginated order history for the caller, newest first. Optional status filter.

Success — 200 OK

```
{
  "items": [
    { "orderId": "8f2a...", "orderNumber": "RDA-2026-000123",
      "status": "CONFIRMED", "grandTotal": "81.93", "currency": "USD",
      "itemCount": 2, "placedAt": "2026-08-19T10:15:00Z",
      "cancellable": true }
  ],
  "page": 0, "size": 20, "totalElements": 8, "totalPages": 1
}
```

cancellable is computed server-side from the 48-hour window and the current status, so the UI never has to reimplement the rule.

GET /api/v1/orders/{id}

Bearer

Full order detail including line items, the shipping-address snapshot, payment status and the status history.

Success — 200 OK

```
{
  "orderId": "8f2a...", "orderNumber": "RDA-2026-000123",
  "status": "CONFIRMED",
  "items": [
    { "bookId": "4c7e...", "title": "Clean Code",
      "isbn13": "9780132350884", "qty": 2,
      "unitPrice": "38.50", "lineTotal": "77.00" }
  ],
  "shippingAddress": { "recipientName": "Pavan B N",
    "line1": "12 MG Road", "city": "Bengaluru",
    "postalCode": "560001", "countryCode": "IN" },
  "payment": { "status": "CAPTURED", "method": "CARD",
    "capturedAt": "2026-08-19T10:15:06Z" },
  "history": [
    { "toStatus": "PENDING_PAYMENT", "at": "2026-08-19T10:15:00Z" },
    { "toStatus": "CONFIRMED", "at": "2026-08-19T10:15:07Z" }
  ],
  "cancellable": true,
  "grandTotal": "81.93", "currency": "USD"
}
```

Error responses

Status	Error code	When it happens
404	ORDER_NOT_FOUND	No such order, or it belongs to another user.

POST `/api/v1/orders/{id}/cancel`

Bearer

Cancels an order. Permitted only within 48 hours of `placedAt` and only while the order has not shipped. Triggers an asynchronous refund and wallet reversal.

Request body

```
{ "reason": "Ordered the wrong edition" }
```

Success — 200 OK

```
{
  "orderId": "8f2a...",
  "status": "CANCELLED",
  "refund": { "status": "PENDING", "amount": "81.93" },
  "cancelledAt": "2026-08-19T18:02:11Z"
}
```

Error responses

Status	Error code	When it happens
404	ORDER_NOT_FOUND	No such order for this caller.
409	ORDER_CANCEL_WINDOW_EXPIRED	More than 48 hours have passed since the order was placed.
409	ORDER_ALREADY_SHIPPED	The order has already shipped and can no longer be cancelled.
409	ORDER_ALREADY_CANCELLED	The order is already in CANCELLED.

The window is evaluated server-side against the stored `placedAt`. The client's clock and the cancellable flag are conveniences only, never the authority.

POST `/api/v1/orders/{id}/reorder`

Bearer

“Buy It Again” — copies a past order's line items into the caller's current cart at **today's** prices and availability.

Success — 200 OK

```
{
  "itemCount": 2, "subtotal": "79.00",
  "skipped": [ { "bookId": "9a1c...", "reason": "OUT_OF_PRINT" } ],
  "priceChanges": [ { "bookId": "4c7e...", "was": "38.50", "now": "39.50" } ]
}
```

Error responses

Status	Error code	When it happens
404	ORDER_NOT_FOUND	No such order for this caller.

Unavailable titles are reported in skipped rather than failing the whole request, and price movements are surfaced explicitly so the user is never silently re-charged a different amount.

payment-service

Payment authorization, capture, refund, wallet debit.

GET

/api/v1/payments/{orderId}

Bearer

Payment status for an order, including refund state when one exists.

Success — 200 OK

```
{
  "paymentId": "p-77a1...", "orderId": "8f2a...",
  "status": "CAPTURED", "method": "CARD",
  "amount": "81.93", "currency": "USD",
  "walletAmountUsed": "50.00",
  "authorizedAt": "2026-08-19T10:15:05Z",
  "capturedAt": "2026-08-19T10:15:06Z",
  "attempts": 1,
  "refund": null
}
```

Error responses

Status	Error code	When it happens
404	PAYMENT_NOT_FOUND	No payment recorded for that order yet — normal in the seconds right after checkout.
403	FORBIDDEN	The order belongs to another user.

Payments are created by the order.created consumer, not by the frontend. There is deliberately no public POST /payments.

ai-service

Spring AI agent, RAG over pgvector, internal tools, MCP client.

POST

/api/v1/ai/chat

Bearer

Sends a message to the assistant. Responds with **streamable HTTP** — a text/event-stream of tokens as the model produces them, rather than one buffered reply. The agent may call internal tools and MCP tools before answering.

Request body

```
Accept: text/event-stream

{
  "conversationId": "5a71...", // omit to start a new conversation
  "message": "where is my Clean Code order?"
}
```

Success — 200 OK (text/event-stream)

```
event: token
data: {"delta":"Your order "}

event: token
data: {"delta":"RDA-2026-000123 "}

event: tool
data: {"name":"getOrderHistory","status":"ok"}

event: done
data: {"conversationId":"5a71...","finishReason":"stop","tokens":214}
```

Error responses

Status	Error code	When it happens
400	VALIDATION_FAILED	Empty message, or message above the configured character limit.
404	CONVERSATION_NOT_FOUND	The conversationId does not exist or belongs to another user.
422	AI_CONTEXT_T00_LARGE	Conversation plus retrieved documents exceed the model context window.
429	AI_RATE_LIMITED	Per-user assistant quota exhausted. Separate from the gateway limit.
502	MCP_TOOL_FAILED	An MCP tool returned an error or timed out; the turn is abandoned rather than answered from a guess.
503	AI_UPSTREAM_UNAVAILABLE	Bedrock unreachable or throttling. The rest of the site is unaffected.

A failure here never blocks shopping — the assistant is additive, and every other service works without it.

GET

/api/v1/ai/search

Public

Semantic catalogue search. Embeds the query and returns the nearest books by cosine similarity over Book.embedding. No LLM call, no agent loop — pure retrieval, so it is fast and cheap enough to sit behind the normal search box. Query params: q, limit (default 10, max 50).

Success — 200 OK

```
{
  "query": "books about writing maintainable code",
  "items": [
    { "bookId": "4c7e...", "title": "Clean Code",
      "authors": ["Robert C. Martin"], "listPrice": "38.50",
      "score": 0.891 },
    { "bookId": "8b2f...", "title": "The Pragmatic Programmer",
      "authors": ["Andrew Hunt", "David Thomas"], "listPrice": "42.00",
      "score": 0.864 }
  ]
}
```

Error responses

Status	Error code	When it happens
400	VALIDATION_FAILED	Missing q, or limit above 50.
503	AI_UPSTREAM_UNAVAILABLE	Embedding endpoint unreachable. Callers fall back to keyword search on catalog-service.

GET

/api/v1/ai/conversations

Bearer

Lists the caller's saved assistant conversations, newest first.

Success — 200 OK

```
{
  "items": [
    { "conversationId": "5a71...", "title": "Clean Code order status",
      "messageCount": 6, "updatedAt": "2026-08-20T09:12:44Z" }
  ],
  "page": 0, "size": 20, "totalElements": 3, "totalPages": 1
}
```

mcp-server

Exposes read-only catalog, commerce and user tools over MCP.

mcp-server does not expose a REST API. It speaks the Model Context Protocol over a streamable HTTP transport at POST /mcp, and is reachable only from inside the Docker network — **not** through the public gateway. Its tool surface is specified in AI Service Architecture.

Shared error envelope

Every service returns errors in this shape, so the frontend has exactly one error path.

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "error": "ORDER_CANCEL_WINDOW_EXPIRED",
  "message": "Orders can only be cancelled within 48 hours of being placed.",
  "status": 409,
  "path": "/api/v1/orders/8f2a.../cancel",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "timestamp": "2026-08-21T18:02:11Z"
}
```

A 400 VALIDATION_FAILED additionally carries per-field detail:

```
{
  "error": "VALIDATION_FAILED",
  "message": "Request contains invalid fields.",
  "status": 400,
  "fieldErrors": [
    { "field": "email", "message": "must be a well-formed email address" },
    { "field": "password", "message": "must be at least 10 characters" }
  ],
  "traceId": "4bf92f3577b34da6...",
  "timestamp": "2026-08-21T18:02:11Z"
}
```

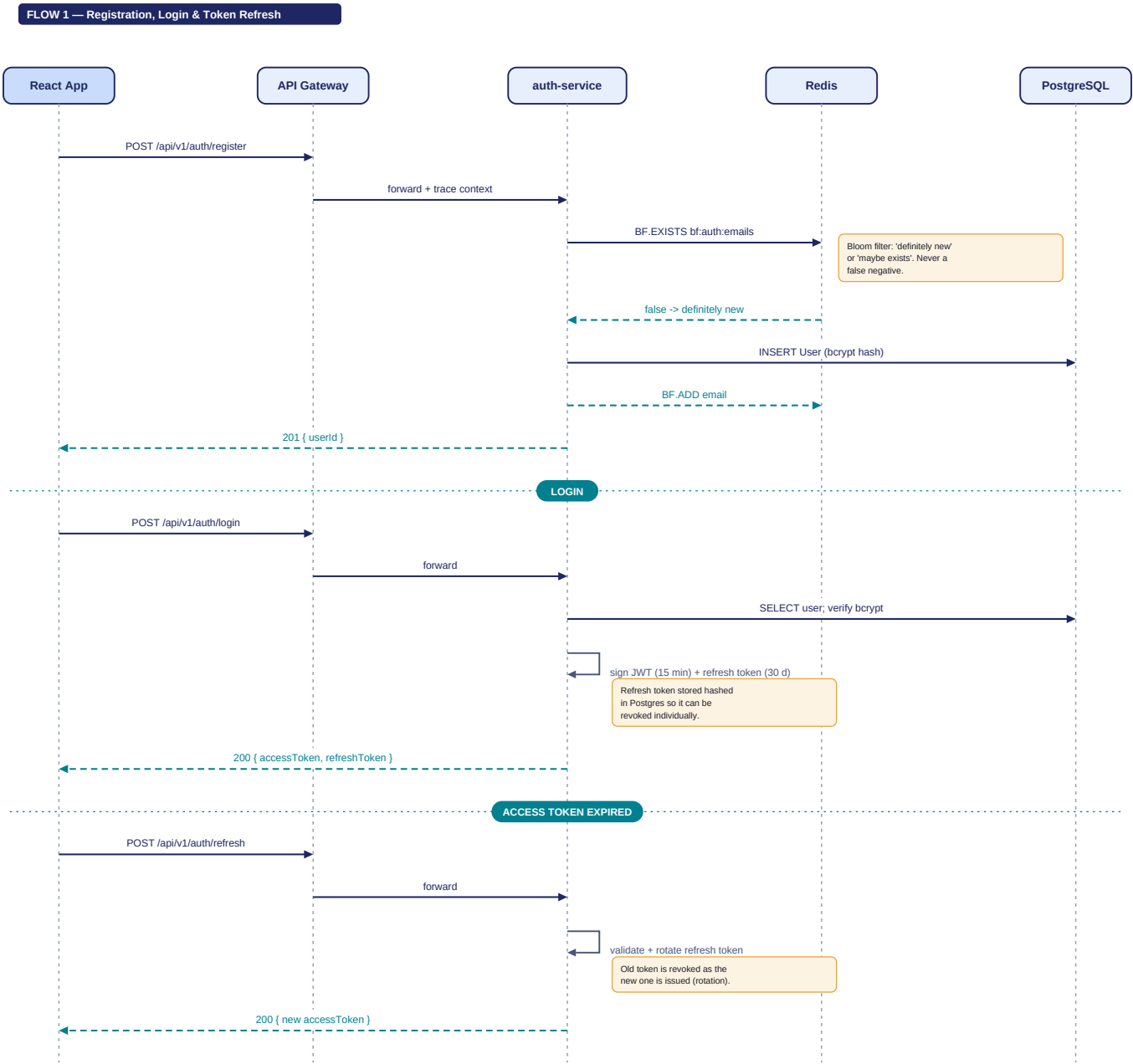
Status	Meaning in Readora
200 / 201 / 204	Success. 201 always returns the created resource; 204 is used only for deletes and logout.
400	The request itself is malformed or fails validation.
401	No token, expired token, or invalid signature.
403	Authenticated, but the resource belongs to someone else.
404	Does not exist, or is not visible to this caller — the two are deliberately indistinguishable.
409	Valid request that conflicts with current state: out of stock, cancel window expired, duplicate email.
422	Understood and well-formed, but semantically impossible: using more wallet balance than available.
429	Rate limit exceeded. Carries a Retry-After header.
500	Unhandled fault. No internal detail is returned; the traceId is how the request is found in New Relic.

Error code reference

Error code	Status	Raised by
VALIDATION_FAILED	400	all services
INVALID_QUERY_PARAM	400	catalog-service
INVALID_CREDENTIALS	401	auth-service
REFRESH_TOKEN_INVALID	401	auth-service
REFRESH_TOKEN_REUSED	401	auth-service
FORBIDDEN	403	all services
ACCOUNT_LOCKED	423	auth-service
BOOK_NOT_FOUND	404	catalog-service, commerce-service
ADDRESS_NOT_FOUND	404	user-service, commerce-service
CART_ITEM_NOT_FOUND	404	commerce-service
GUEST_CART_NOT_FOUND	404	commerce-service
ORDER_NOT_FOUND	404	commerce-service
PAYMENT_NOT_FOUND	404	payment-service
CONVERSATION_NOT_FOUND	404	ai-service
EMAIL_ALREADY_REGISTERED	409	auth-service
ADDRESS_LIMIT_REACHED	409	user-service
CART_EMPTY	409	commerce-service
INSUFFICIENT_STOCK	409	commerce-service
PRICE_CHANGED	409	commerce-service
ORDER_CANCEL_WINDOW_EXPIRED	409	commerce-service
ORDER_ALREADY_SHIPPED	409	commerce-service
ORDER_ALREADY_CANCELLED	409	commerce-service
QTY_LIMIT_EXCEEDED	422	commerce-service
INSUFFICIENT_WALLET_BALANCE	422	commerce-service
AI_CONTEXT_TOO_LARGE	422	ai-service
RATE_LIMIT_EXCEEDED	429	api-gateway
AI_RATE_LIMITED	429	ai-service
MCP_TOOL_FAILED	502	ai-service
AI_UPSTREAM_UNAVAILABLE	503	ai-service

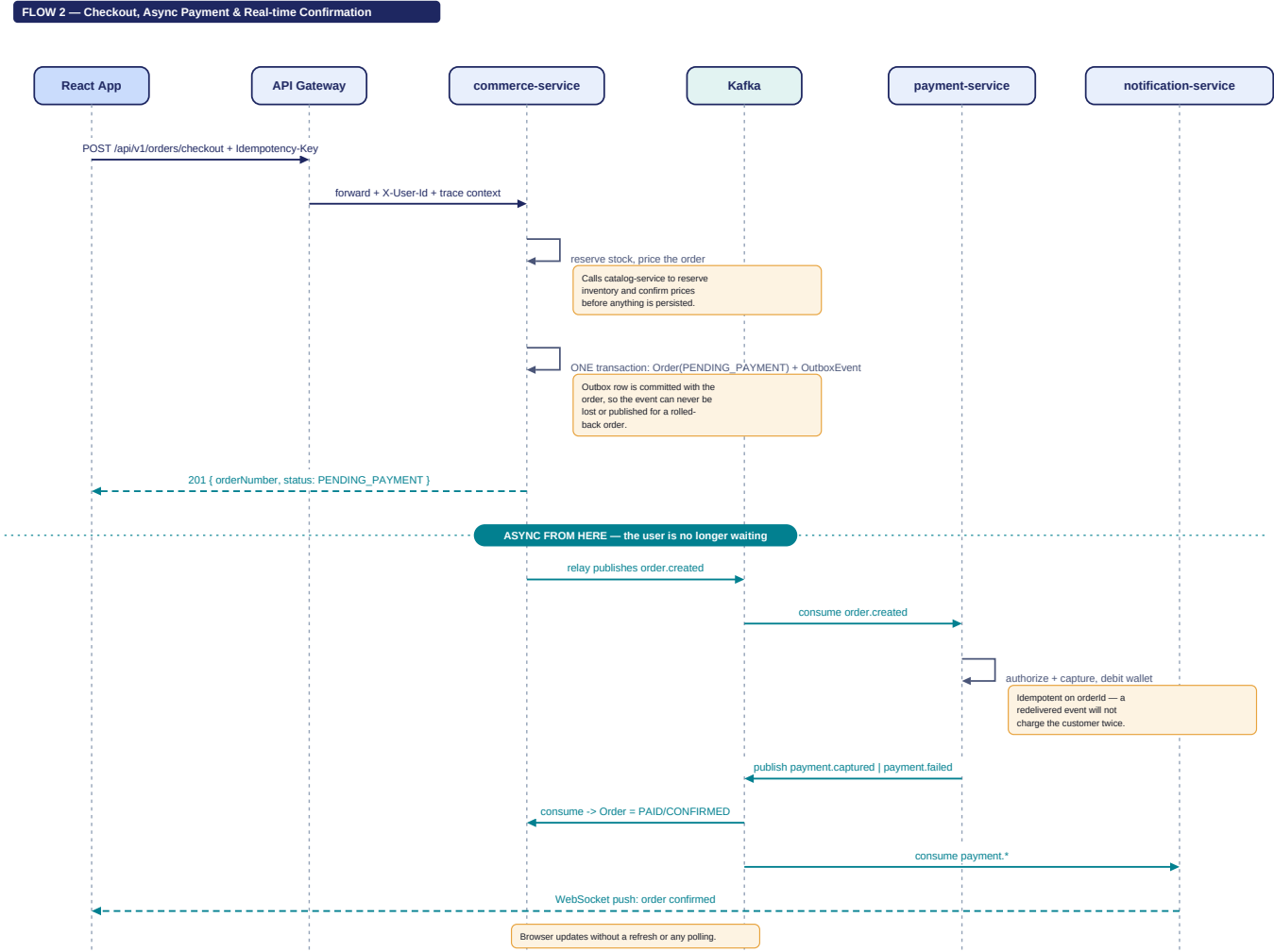
Sequence Diagrams

Registration, login and token refresh



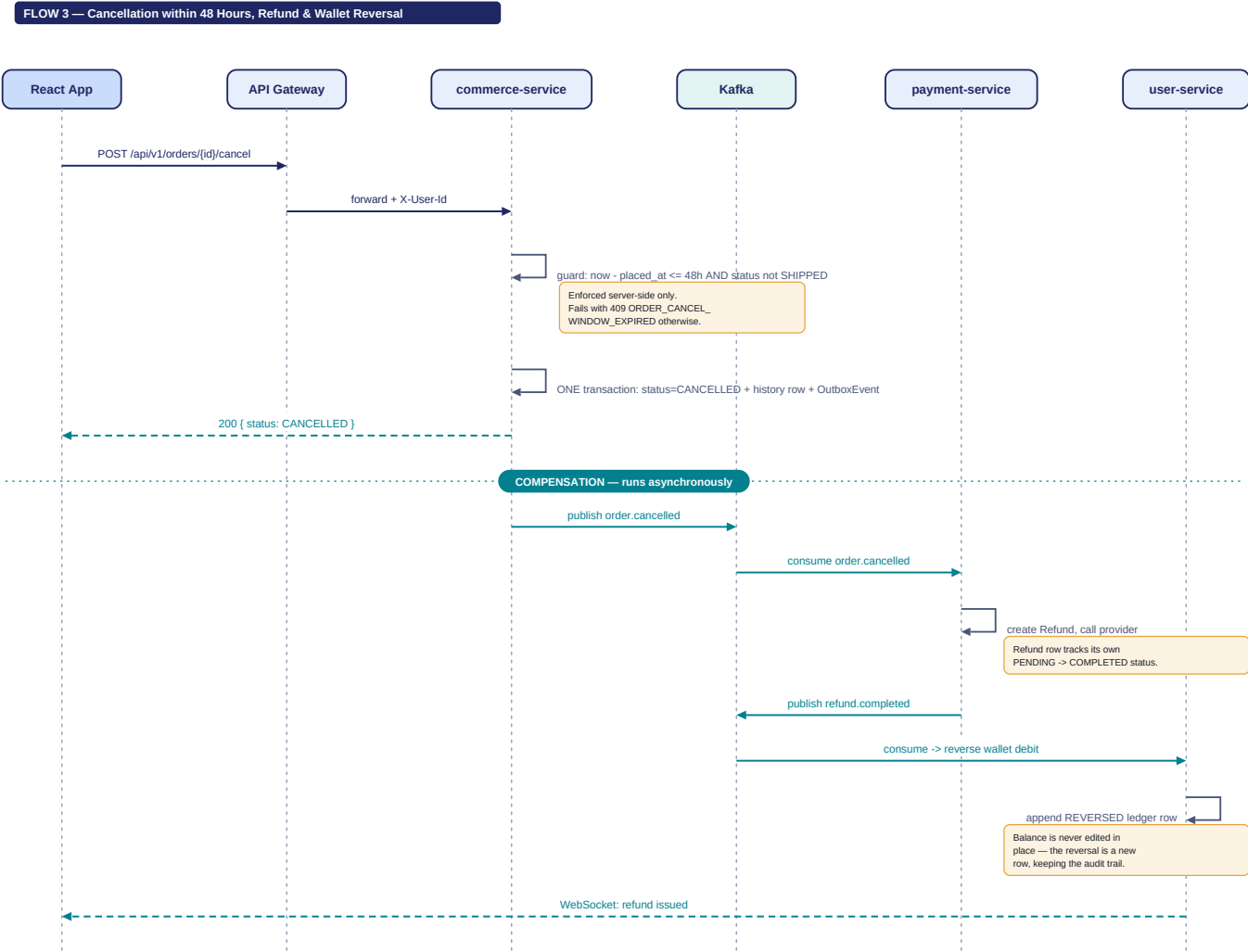
Identity flows, including the Bloom-filter pre-check and refresh-token rotation.

Checkout, asynchronous payment and real-time confirmation



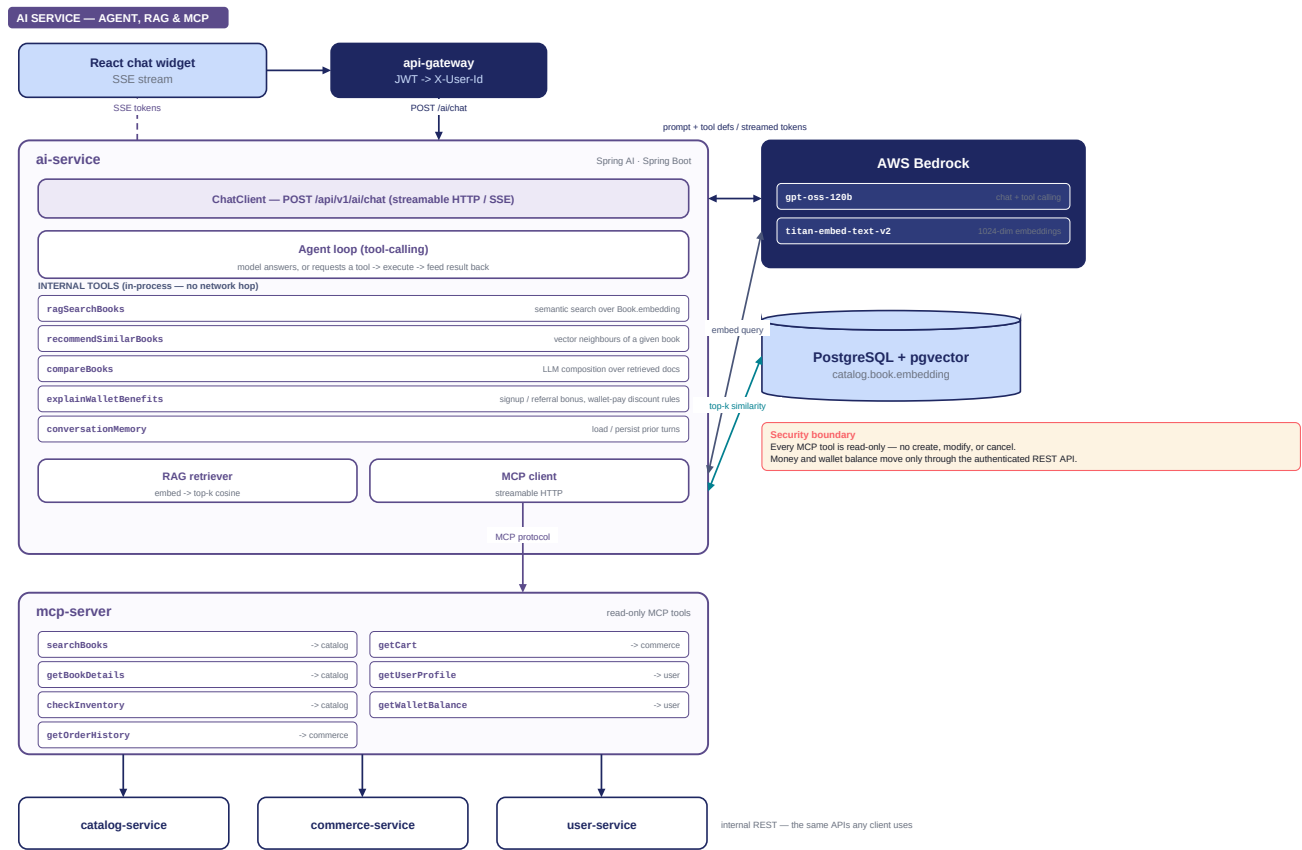
The central flow. commerce-service and payment-service never call each other directly.

Cancellation, refund and wallet reversal



The 48-hour rule and the compensating actions it triggers.

AI Service Architecture



ai-service internals, the model provider, and the MCP tool surface.

Internal tools

Executed in-process by ai-service. They operate on its own vector store and on the model — no other service is involved and no network boundary is crossed.

Tool	Arguments	What it does
ragSearchBooks	query, limit	Embeds the query with Titan V2 and returns the nearest books by cosine similarity over Book.embedding.
recommendSimilarBooks	bookId, limit	Vector neighbours of a given book — “more like this” without a keyword overlap.
compareBooks	bookIds[]	Composes a comparison from documents already retrieved into context.
explainWalletBenefits	—	Answers signup-bonus, referral-bonus and wallet-pay discount questions from grounded rules.
conversationMemory	conversationId	Loads and persists prior turns so a follow-up question keeps its context.

MCP tools

Exposed by mcp-server and called by the agent through its MCP client. Each one reads live data owned by another service over that service's normal REST API, scoped to the caller's X-User-Id. **All are read-only.**

Tool	Arguments	Downstream	Returns
searchBooks	query, filters	catalog-service	Matching books with price and availability

Tool	Arguments	Downstream	Returns
getBookDetails	bookId	catalog-service	Full book record including format and publisher
checkInventory	bookId	catalog-service	Available quantity and stock status
getOrderHistory	limit, status	commerce-service	The caller's orders, newest first
getCart	—	commerce-service	Current cart contents and subtotal
getUserProfile	—	user-service	Display name, locale, default address city
getWalletBalance	—	user-service	Current balance and recent ledger entries

The agent cannot change anything

There is deliberately no `addToCart`, `placeOrder` or `cancelOrder` tool. Every MCP tool is a read. Mutations stay behind the authenticated REST API and are driven by the user's own click, never by model output — so a prompt injection in a book description cannot cause a purchase, a cancellation or a wallet spend.

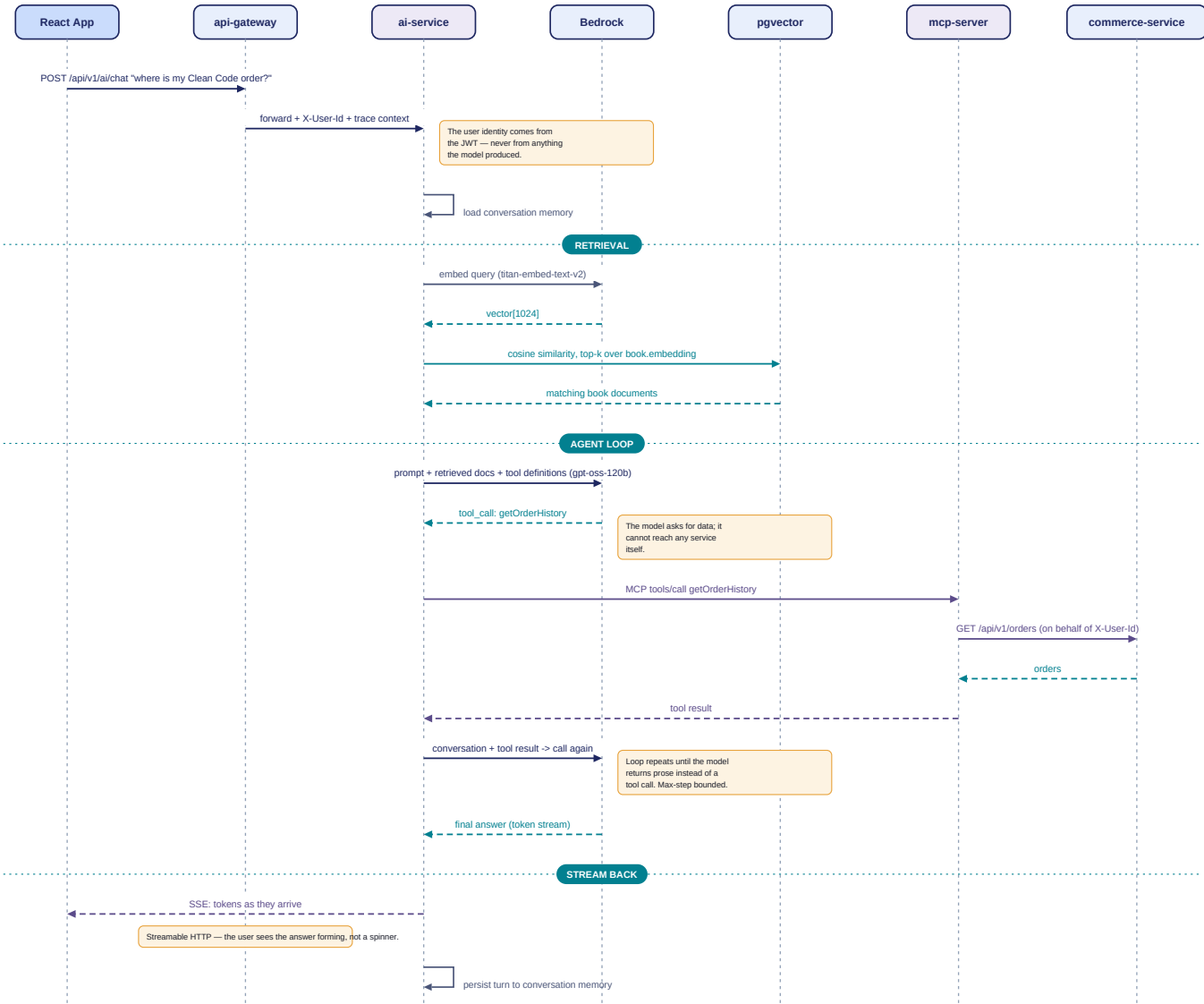
Model and transport

Concern	Choice
Framework	Spring AI — <code>ChatClient</code> , tool-calling, and the MCP client integration
Chat model	gpt-oss-120b on Amazon Bedrock (exact model id set in configuration, not compiled in)
Embedding model	Amazon Titan Text Embeddings V2 , 1024 dimensions, on the same Bedrock credentials
Vector store	pgvector on the existing Postgres container, via Spring AI <code>PgVectorStore</code>
Client transport	Streamable HTTP — <code>text/event-stream</code> of tokens to the browser
Agent ↔ tools	MCP over streamable HTTP to <code>mcp-server</code> ; internal tools in-process
Loop control	Bounded max tool-call steps per turn, plus a per-user request quota

`gpt-oss-120b` is a text-generation model and cannot produce embeddings — the separate Titan embedding model is required, not an alternative. Both run on Bedrock so there is one credential path and one SDK.

Agent request lifecycle

FLOW 4 — AI Agent: RAG Retrieval, MCP Tool Call & Streamed Answer



One assistant turn: retrieval, an MCP tool call, and the streamed answer.

Eventing

Topics

Topic	Producer	Consumers	Payload
order.created	commerce-service	payment-service	orderId, userId, items[], grandTotal, walletAmountToUse, paymentMethod
order.cancelled	commerce-service	payment-service, notification-service	orderId, userId, reason, refundAmount
payment.captured	payment-service	commerce-service, user-service, notification-service	orderId, paymentId, amount, walletAmountUsed

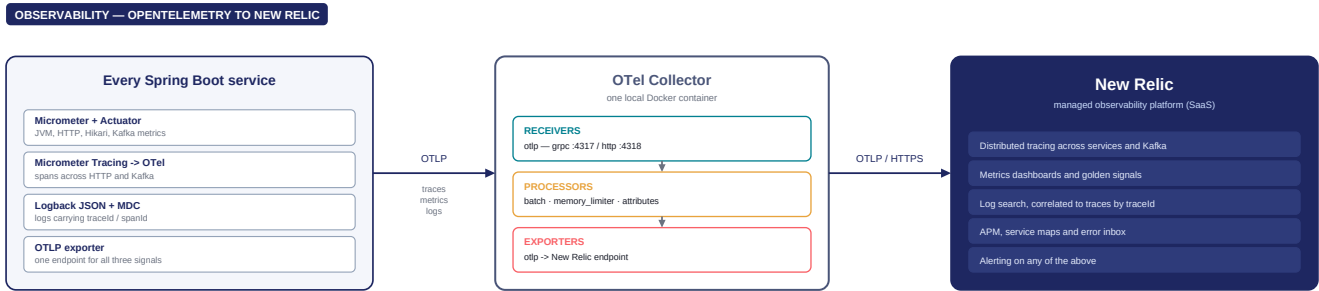
Topic	Producer	Consumers	Payload
payment.failed	payment-service	commerce-service, notification-service	orderId, failureCode, failureReason
refund.completed	payment-service	user-service, notification-service	orderId, refundId, amount, walletAmountToReverse

Delivery guarantees

Concern	Approach
Lost events	Transactional outbox — the event row commits with the business row; a relay publishes afterwards.
Duplicate delivery	Every consumer is idempotent, keyed on orderId or the event's eventId. At-least-once is assumed.
Ordering	Messages are keyed by orderId, so all events for one order land on the same partition and stay ordered.
Poison messages	Retry with backoff, then route to <topic>.DLT. A dead-letter message alerts rather than blocking the partition.
Traceability	W3C trace context travels in Kafka headers, so async work joins the trace of the request that caused it.

Observability

Every service emits **traces, metrics and logs** through OpenTelemetry, exports them to a single collector, and forwards them to **New Relic**.



One instrumentation standard, one collector, one managed backend.

Instrumentation

Signal	Produced by	Answers
Traces	Micrometer Tracing bridged to OpenTelemetry; auto-instruments Spring MVC, WebClient and Kafka listeners	Where did the time go, across services and across the Kafka boundary?
Metrics	Micrometer via Spring Boot Actuator, exported over OTLP	Is the system healthy right now — rate, errors, latency?
Logs	Logback JSON encoder with traceId and spanId injected from the MDC	What exactly happened inside this one request?

What is measured

Category	Metrics
RED (per service, per endpoint)	request rate, error rate, duration percentiles (p50/p95/p99)
JVM	heap and non-heap usage, GC pause time, live thread count
Datasource	HikariCP active/idle/pending connections, connection acquisition time
Kafka	consumer lag per group, records consumed/sec, retry and DLT counts
Redis	cache hit ratio, command latency, evicted keys
AI	tokens per turn, tool-call count, Bedrock latency and error rate, pgvector query time
Business	orders placed, checkout conversion, payment success rate, cancellations within 48h

Health is exposed at `/actuator/health` with separate liveness and readiness groups.

Security

Control	Where	Detail
JWT authentication	api-gateway + every service	Gateway validates signature and expiry, then injects X-User - Id. Each service re-validates independently — the header alone is never trusted.
Refresh-token rotation	auth-service	Tokens stored hashed and revocable. Reuse of a revoked token revokes the whole session family.
Password storage	auth-service	bcrypt with a per-user salt. Plaintext never persisted or logged.
Rate limiting	api-gateway	Token bucket (Bucket4j + Redis), per user when authenticated and per IP otherwise. Returns 429 with Retry-After.
Bloom filters	auth-service, catalog-service	Cheap negative lookups: unregistered emails and never-existed book ids are answered without touching Postgres.
Authorization	each service	Ownership is checked on every user-scoped resource. Another user's order returns 404, not 403 — existence is not leaked.
Idempotency	commerce-service, payment-service	Unique idempotency keys make retried checkouts and redelivered payment events safe.
AI: read-only tools	mcp-server	No tool can create, modify or cancel anything. Model output can never cause a purchase, cancellation or wallet spend.
AI: identity from JWT	ai-service, mcp-server	Every tool call is scoped to the authenticated X-User - Id. The model cannot name a different user and read their data.
AI: injection containment	ai-service	Retrieved book text is untrusted data, not instructions. Because the tool surface is read-only and user-scoped, an injected instruction has nothing harmful to reach.
AI: cost + loop bounds	ai-service	Max tool-call steps per turn and a per-user quota; a runaway plan cannot burn tokens indefinitely.
Secrets	docker-compose env	DB passwords, JWT signing key, AWS credentials and the New Relic licence key are injected as environment variables, never committed.

Containerization & CI

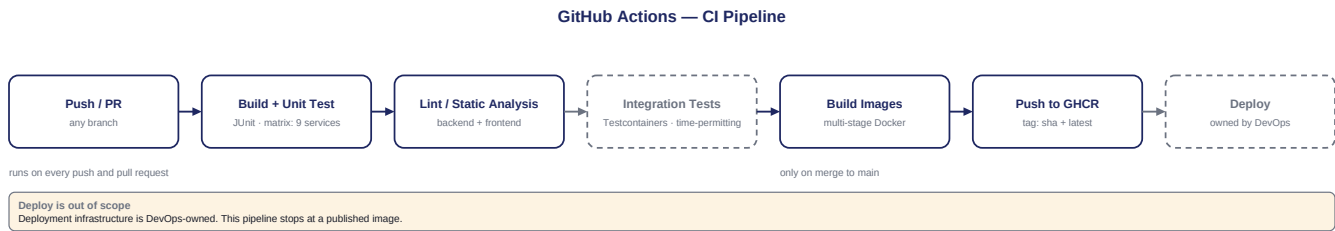
Images

Image	Build stage	Runtime stage
*-service, mcp-server	maven:3-eclipse-temurin-21 → mvn package	eclipse-temurin:21-jre-alpine
frontend	node:20-alpine → npm run build	nginx:alpine serving the static bundle

Testing strategy

Layer	Priority	Detail
Unit tests (JUnit)	Required every phase	Written alongside each service as it's built — not deferred.
Integration tests (Testcontainers)	Time-permitting	Real Postgres (with pgvector), Redis and Kafka. No H2 or embedded broker.
End-to-end tests	Time-permitting	Full journey through the gateway; lowest priority given remaining time.

Pipeline



GitHub Actions. Everything through image push is real; deploy is out of scope.

Delivery Phases

#	Phase	Exit criteria	Status
1	Foundations	docker compose brings up Postgres, Redis, Kafka; a skeleton service answers /actuator/health	Planned
2	Core services	Register → browse → cart → checkout → pay → order history works end to end locally	Planned
3	Security & observability	One trace followed end to end in New Relic; logs correlated by traceId; 429s under load	Planned
4	Async & real-time	Checkout returns immediately; confirmation arrives via WebSocket; the trace spans the Kafka boundary	Planned
5	AI & MCP	Embeddings backfilled; RAG search returns relevant books; the agent answers an order question through an MCP tool and streams the reply	Planned — last backend phase
6	Frontend completion	Full journey plus the chat widget usable at desktop and mobile widths with no console errors	Planned
7	Containerization	A clean clone runs the entire system with one docker compose up	Planned
8	CI	A pull request produces a green pipeline and a published image, no manual steps	Planned

Deployment infrastructure — cloud target, Kubernetes or otherwise — is DevOps/SRE ownership, not part of this repository's scope.